



Introduction to Kubeflow

Kubernetes-Native Platform for MLOps at Scale

A comprehensive platform that streamlines machine learning workflows by leveraging Kubernetes for orchestration, scalability, and reproducibility.

Why Kubeflow?

Scalable Orchestration

ML workflows require robust, scalable infrastructure beyond what ad-hoc scripts can provide

Kubernetes Foundation

Leverages containers & microservices architecture for flexibility and resilience

Unified Platform

Single solution for data processing, model training, and deployment workflows



Core Goals of Kubeflow



Portability

Run consistent ML infrastructure across any cloud provider or on-premises environment without code changes



Scalability

Effortlessly scale from experiments on a single GPU to production workloads on thousands of nodes



Reproducibility

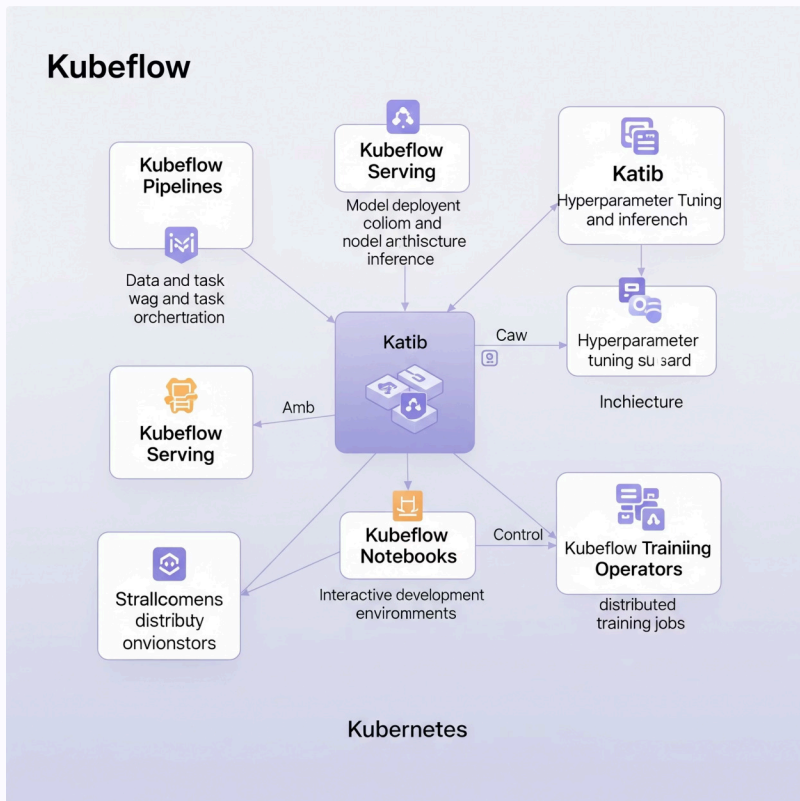
Version pipelines, artifacts, and configurations to ensure consistent, reproducible workflows



Integration

Works seamlessly with TensorFlow, PyTorch, scikit-learn, XGBoost, and Hugging Face models

Key Components



Kubeflow Pipelines (KFP)

Workflow automation & experiment tracking

KServing (KServe)

Model deployment with intelligent autoscaling

Katib

Hyperparameter tuning at scale

Notebooks

JupyterHub integration for experimentation

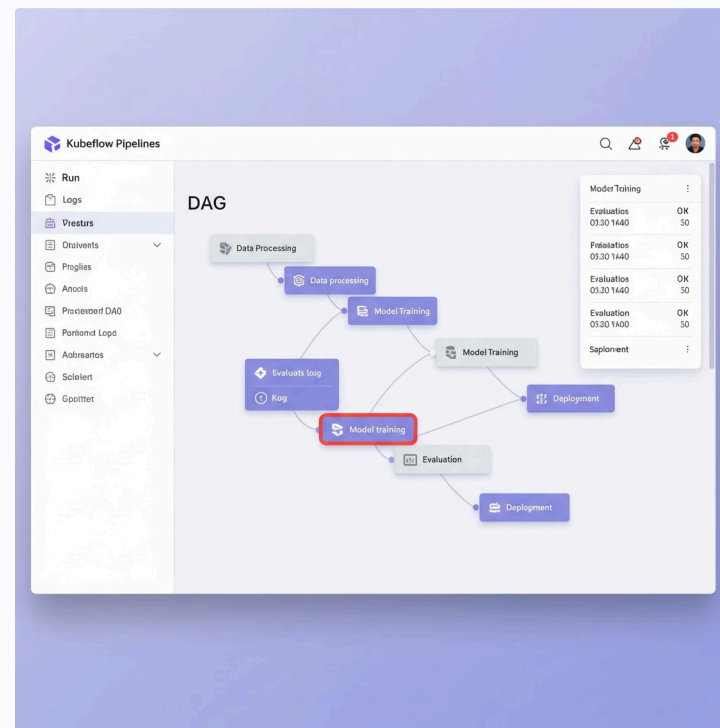
Training Operators

PyTorchJob, TFJob, MPIJob for distributed training

Kubeflow Pipelines (KFP)

KFP provides a powerful framework for creating reproducible, parameterizable ML workflows:

- DAG-based pipeline structure from data ingestion to deployment
- Comprehensive artifact tracking and metadata lineage
- Easy parameterization for rapid experimentation
- Rich UI for tracking runs and comparing results



Pro Tip: Modularize pipeline components for reusability across multiple workflows



Training on Kubeflow



PyTorchJob

Distributed training for PyTorch models with elastic GPU allocation



TFJob

Native TensorFlow cluster management with parameter servers and workers



MPIJob

MPI-based operator for large-scale distributed deep learning workloads

Kubeflow automatically handles the complex orchestration of scheduling, retries, and efficient GPU allocation across your Kubernetes cluster.

Model Serving with KServe

Microservice-Based Architecture

- Deploy models as scalable API endpoints on Kubernetes
- Support for both REST and gRPC inference protocols
- Knative integration for intelligent autoscaling (scale-to-zero)
- Multi-framework serving for TorchScript, TensorFlow, ONNX, and Triton



❏ KServe automatically handles traffic splitting for canary deployments and A/B testing of new model versions

Hyperparameter Tuning with Katib



Define Experiment

Specify parameter ranges, objective metrics, and search algorithm



Run Distributed Trials

Katib schedules trials across your cluster with Bayesian, grid, random, or evolutionary search



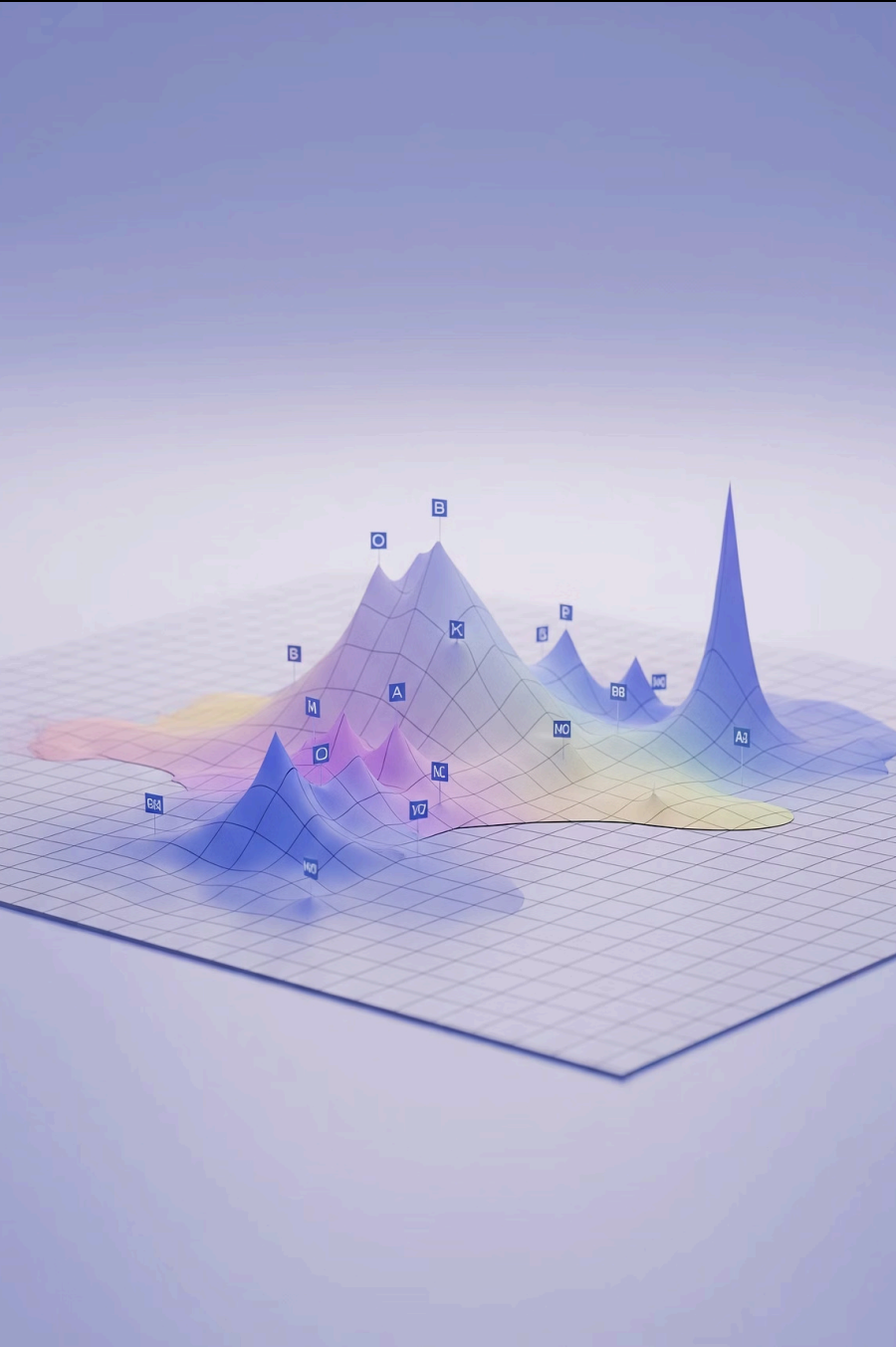
Track & Select Best

Metrics are logged automatically for visualization and best-trial selection



Automate Retraining

Integrate with Pipelines for continuous optimization loops



Security & Multi-Tenancy

Namespace Isolation

Separate users and projects with Kubernetes namespaces for resource isolation

Role-Based Access Control

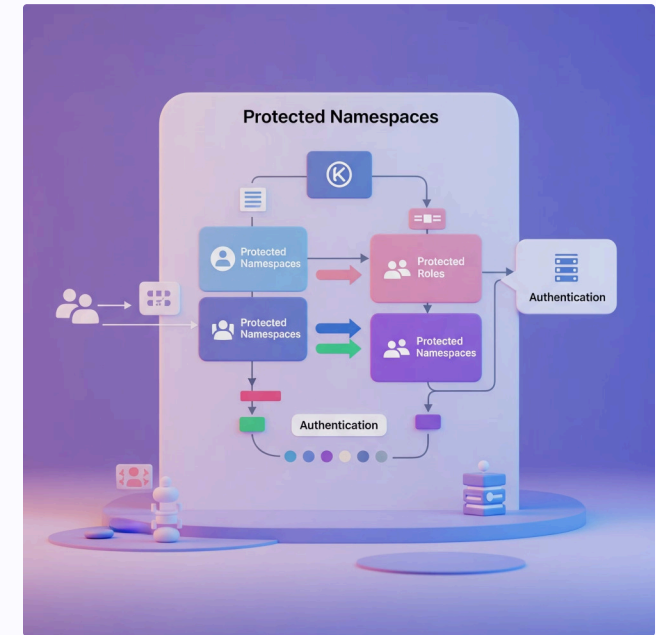
Leverage Kubernetes RBAC for fine-grained permission management

Identity Integration

Connect to enterprise identity systems via OIDC, LDAP, and other protocols

Shared Infrastructure

Enable multiple teams to use the same cluster with appropriate guardrails



Kubeflow in Action

A unified platform for the entire ML lifecycle across any infrastructure:

Data Preparation
Ingest, transform, and validate
training data

Monitoring
Track performance and detect
model drift



Model Training

Scale across CPUs/GPUs with
distributed operators

Tracking

Monitor metrics, artifacts, and
lineage

Deployment

Serve models with KServe's
autoscaling capabilities

Cloud-agnostic: [GCP \(Vertex AI\)](#) · [AWS \(EKS\)](#) · [Azure \(AKS\)](#) · On-Prem (OpenShift)

Best Practices

Use KFP for Reproducibility

Replace ad-hoc scripts with versioned, parameterized pipelines for consistent experimentation

Persistent Storage Strategy

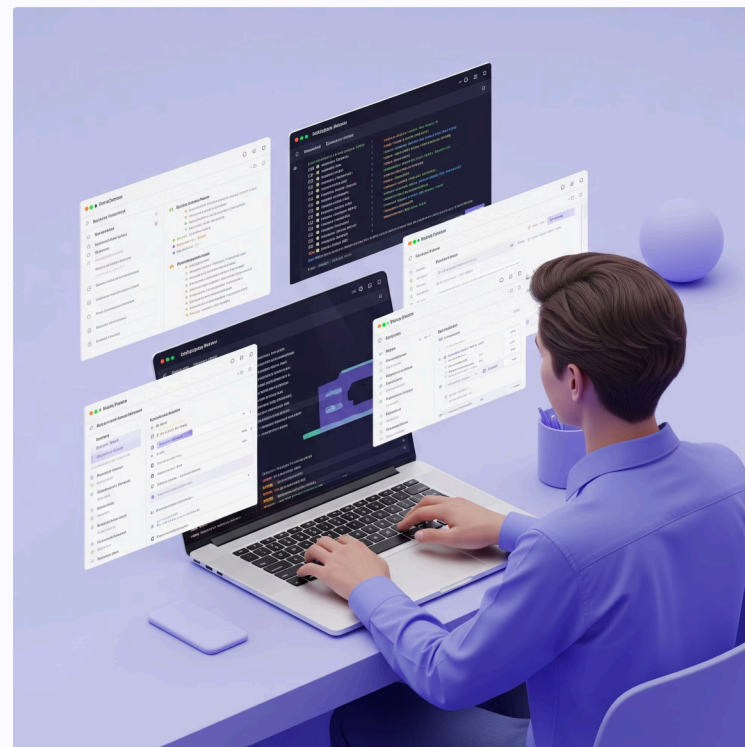
Store models and artifacts in persistent volumes or cloud storage (S3/GCS) for durability

Efficient Inference with KServe

Deploy with autoscaling to balance performance and cost—scale to zero when idle

Automate with Katib

Leverage hyperparameter optimization to create continuous search → retrain loops

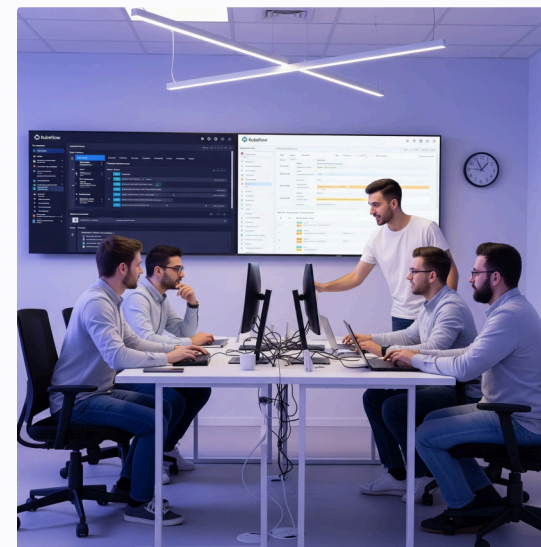


Common Pitfall: Neglecting to set resource limits can lead to cluster instability under heavy workloads

Key Takeaways

Kubeflow: End-to-End ML Platform on Kubernetes

- Complete orchestration for training, pipelines, serving, and tuning
- Portable across all cloud providers and on-premises environments
- Designed for enterprise MLOps with security and multi-tenancy
- Start incrementally: adopt components based on your immediate needs



Get Started: github.com/kubeflow/kubeflow